

A PRACTICAL GUIDE TO AI AGENTS

Building AI Agents from Scratch

A Hands-On Guide with Python



Yanfei Li

Building AI Agents from Scratch

A Hands-On Guide with Python

Yanfei Li

First edition · 2026

Building AI Agents from Scratch

A Hands-On Guide with Python

Copyright © 2026 Yanfei Li. All rights reserved.

First edition, 2026.

The book text and cover may not be reproduced or redistributed without permission, except as permitted by applicable law. Companion source code is distributed under the MIT License included in the code package.

Product names and trademarks belong to their respective owners. This book is independently published and is not endorsed by the providers discussed.

Examples are educational. Some are study fragments; others are complete programs with documented dependencies. Live integrations require your own accounts and may incur charges. Model identifiers and APIs can change.

Code and technical references were reviewed in September 2026. Local checks do not verify every external integration or production deployment.

Companion code

The companion package follows the chapter folders named throughout this book. Refer to the download instructions provided with your purchased edition. Start with the package README and Chapter 3. Code filenames shown in the chapters refer to that package.

About the Author

Yanfei Li, PhD

Yanfei Li is a Senior AI Engineer with a background in energy systems and software development. His work focuses on building AI systems and applying computational methods to solve engineering problems.

He earned a Master of Science in Computer Science with a focus on artificial intelligence from the Georgia Institute of Technology in 2026. He also holds a PhD in Mechanical Engineering, with a focus on modeling, control, and optimization.

Through practical, everyday examples, he aims to make AI agent development accessible and help readers turn ideas into working software.

1 What Are AI Agents?

Suppose a user asks, “What is the hotel limit in our travel policy, and are exceptions allowed?” An agent can search the available documents, read the relevant policy and answer from what it finds. Building that application means connecting a decision process to real operations and carrying their results into the next decision.

What you will learn. Distinguish agents, workflows and chatbots; identify the components of an agent system; follow a request end to end; run a small demonstration of the execution loop.

What you need. Basic Python: functions, dictionaries, loops and running a file in a terminal. Chapter 3 covers environment setup. The demonstration uses local data and needs no model account. Companion folder: `code-examples/chapter-01`.

1.1 Choose the Right Kind of Application

An **agent** observes an environment and chooses actions toward a goal. The environment might be files, business services or physical devices. Russell and Norvig describe this perception–action relationship and tie action selection to a performance measure [1]. For a developer, the useful starting questions are: what can the system observe, what can it do, and what counts as success?

This book builds **LLM agents**: applications in which a large language model selects some next actions from the task and previous results. An **LLM** is a trained model that processes and generates language. The surrounding application supplies context, exposes tools and executes permitted requests.

Table 1.1: Agent, chatbot, script and workflow distinctions

Term	What the term describes	Useful when
Automation script	Code-defined operations and rules.	The required procedure is known and testable.
Workflow	The steps and branches of a process.	The process can be specified explicitly.
LLM workflow	A workflow with model calls at defined points.	A fixed process needs language interpretation or generation.
LLM agent	A system with some model-selected next actions.	Follow-up work depends on varied observations.
Chatbot	A conversational user interface.	People need to ask, clarify, or instruct in conversation.

These terms overlap. A chatbot can expose an agent through a conversation; the same agent can run from a command line or an API. A Python script can

implement either fixed rules or a model-driven loop. Choose the interface and execution design separately.

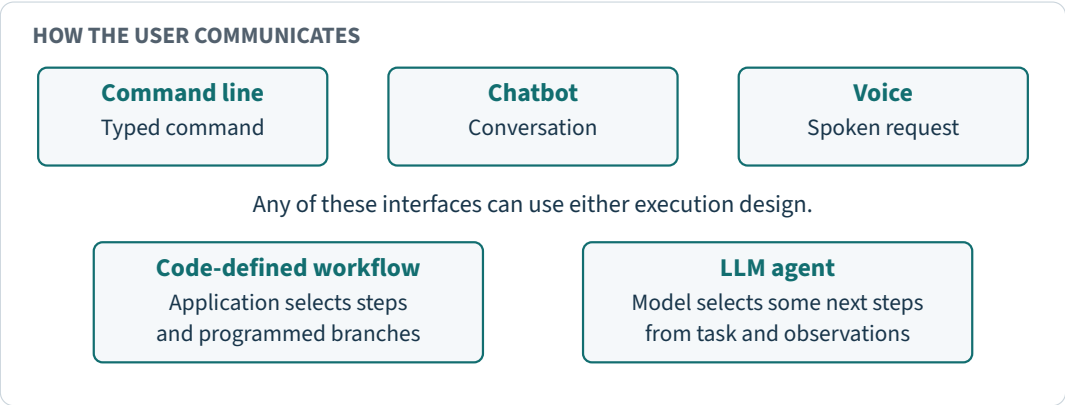


Figure 1.1: Interface and execution are separate choices. A chat interface can sit over either a fixed workflow or a model-directed loop; a command-line interface can do the same.

For the policy question, compare three implementations. A search script returns matching lines. An LLM workflow follows a fixed search-and-summarize sequence. An agent can use a search result to choose which document to read next.

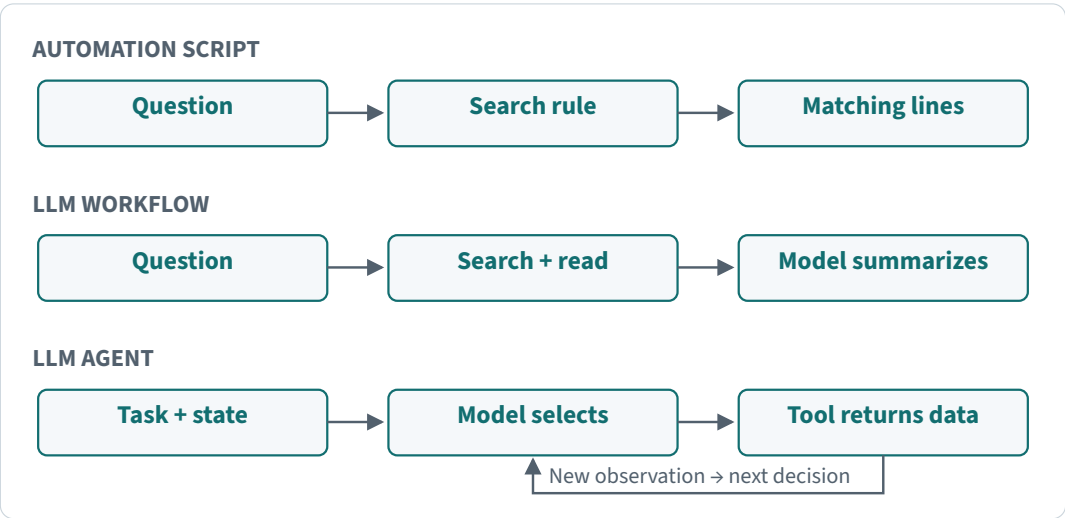


Figure 1.2: Three execution designs for the same document question. In the bottom design, returned evidence can trigger another model-selected operation. The other designs can still contain explicitly programmed error handling.

Anthropic’s engineering guide recommends starting with simple workflows and adding agent control when a task needs flexible follow-up [2]. This gives us a practical selection rule: use known rules for predictable work, a fixed model step for language processing, and an agent loop when observations should influence the next action.

1.2 A General Architecture for Agent Systems

An **architecture** describes components and their connections. The following reference architecture supports one agent or a team. Several components can live in one Python process.

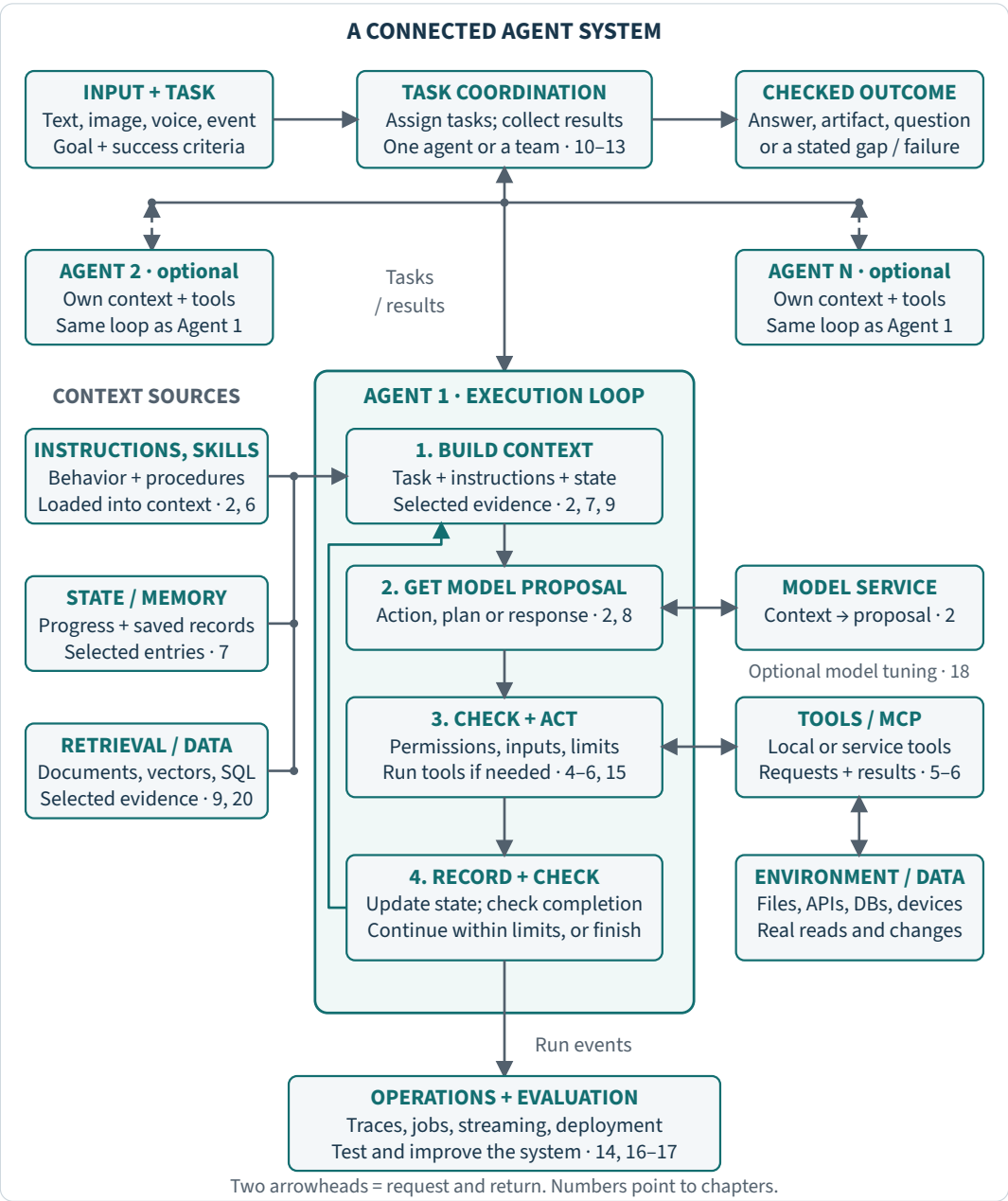


Figure 1.3: A connected agent system. The top row follows a request from task intake to a checked outcome. Agent 1 is expanded to show how context sources, model access and tools connect to its execution loop; other agents use the same structure. Two arrowheads indicate requests and returns. Numbers point to the implementation chapters.

Read the top row first: the application assigns work to one agent or a team, collects results and returns an outcome. Then follow Agent 1 downward. Instructions, state and selected evidence feed its context. The runtime asks the model for a proposal, checks permissions and limits, and runs a tool if needed. It records the result and checks completion. The green return arrow adds new observations to the next iteration; a finished result goes back to task coordination. Run events support monitoring and evaluation.

A **multi-agent system** coordinates multiple agent instances. Each instance has its own instructions, working context and available actions; instances may share a model service. An **orchestrator** routes work and manages dependencies or handoffs. Use a team when separate capabilities, parallel work or independent review offer a measurable benefit. Chapters 10–13 develop those choices.

The component map below explains each responsibility, why it is useful and where to implement it. Start with the task, model, runtime and tools in Chapters 2–5; add the other capabilities when the application needs them.

Table 1.2: System modules, their purpose and chapter map

Module and meaning	Why or when it is needed	Ch.
Model: a service that proposes text or operations from context	Language interpretation and flexible next-step selection; choose by task tests, limits and cost	2
Development environment: Python, dependencies and configuration	Makes the same program installable and its failures diagnosable	3
Intake, runtime and outcome: accept a goal, execute the loop, report how it ended	Every application needs a defined request, execution boundary and stopping condition	4
Tool contract: a named operation with validated inputs and results	Connect decisions to real reads, calculations or changes; grant only needed operations	5
MCP adapter and skill: a shared tool protocol and reusable procedural instructions	Use MCP to connect compatible applications/services; use a skill to teach a recurring procedure	6
State, context and saved memory: retained run data, selected model input and reusable records	Track progress; add durable storage when work must survive a restart or be reused	7
Reasoning and planning: choose actions and arrange dependencies	Use observations to adapt; add explicit plans when unfinished dependencies become hard to track	8
Retrieval and index: select evidence from prepared sources	Ground answers in material too large or changeable to include in every request	9
Team and orchestrator: distinct agents plus assignment and termination rules	Test when separate roles, permissions or parallel work have a measurable benefit	10

Module and meaning	Why or when it is needed	Ch.
Message contract: a defined envelope for task and result exchange	Prevent confusion about sender, task, correlation, retries and completion	11
Framework: reusable runtime or coordination machinery	Adopt when its tested behavior reduces work for your required control flow	12
Delegation and human review: assign bounded responsibility or ask an authorized person	Handle larger tasks or decisions that need human judgment or permission	13
Evaluation and traces: task tests and records of what ran	Detect unsupported answers and regressions; a reviewer agent alone cannot prove correctness	14
Execution controls: permissions, validation, budgets and failure handling	Apply at actual execution boundaries; decide what may run and what happens when it fails	15
Deployment and operations: API, jobs, workers, storage and release procedure	Make work available beyond one terminal and recover interrupted or failed runs	16
Streaming: incremental delivery of text and progress events	Help an interface show progress; partial output must not trigger an incomplete tool call	17
Training: an offline experiment that changes model parameters	Consider only after diagnosing a learnable model behavior and defining held-out evaluation	18
Multimodal intake: text, images or document pages as evidence	Add when required facts live in more than text; check coverage and extraction separately	19
Database access: scoped queries against structured records	Use original typed records when the task needs filters, joins or aggregates	20
Voice interface: speech input/output around the agent loop	Add when speaking helps the task; test recognition, audio devices and interruption separately	21

The diagram's shared services are reusable capabilities. For example, a document agent may need retrieval, while a calculator agent may need only one tool. Model access, stored memory, retrieval and tool execution remain separate responsibilities even when a framework packages them together.

1.2.1 Inside One Agent

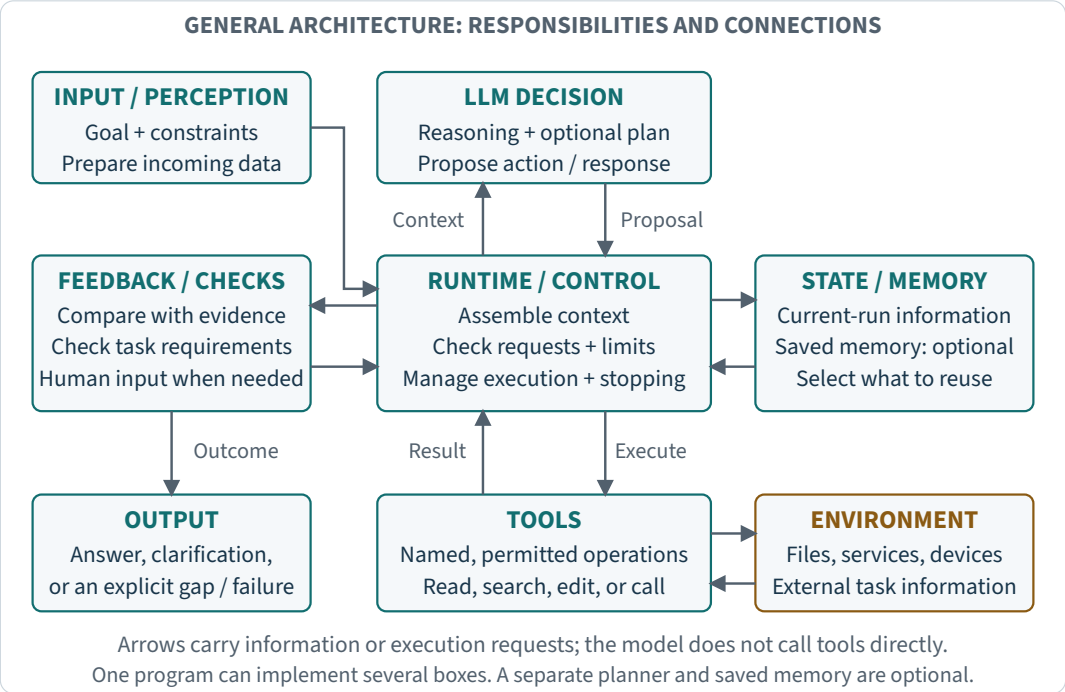


Figure 1.4: General architecture of an LLM agent. Application code prepares inputs and context, calls the model, controls tool execution, records observations, and checks outcomes. Optional saved memory supports later runs. Files, services, and devices remain in the external environment.

The **runtime** is the application code that manages the loop: prepare model input, inspect the proposed action, execute a tool, record its result and decide whether to continue. A **tool** is a named operation the runtime can execute, such as reading a file or querying an API. The model receives a description of that operation and proposes arguments.

State is the information retained during the task: progress, observations and remaining limits. **Context** is the selected information sent in one model request. Saved **memory** makes information available for later work. These distinctions matter because a large stored history need not be sent to the model on every step.

A **goal** states the desired outcome. Instructions and constraints tell the agent how to work toward it. A plan can track dependent steps, but a separate planning component is optional. Anthropic’s guide describes tools, retrieval and memory as capabilities around the model; its recommendation is to choose them for the task [2].

Feedback is information returned after an action, such as retrieved text or an error. **Verification** compares a result with the task’s requirements. For the policy assistant, check the amount, unit, tax condition, exception rule and

source. Model review and human approval are additional options developed in Chapter 13.

1.2.2 Start with the Components You Need

Table 1.3: Choosing the smallest useful implementation

Responsibility	A first document assistant	Add more when
Input and goal	One text question, allowed sources, and answer requirements.	Images, speech, or missing task details require more input handling.
Decision and planning	One model selects the next search, read, or response.	Dependencies and unfinished subtasks require a stored plan.
State and memory	Retain source IDs, observations, and counters for this run.	Restarts or justified reuse require durable records.
Tools and environment	Search and read an allowed source folder.	The task requires other data or permitted actions.
Execution and checks	Validate requests, enforce limits, and compare results with evidence.	External changes require stronger confirmation or approval.

For the first document assistant, two read-only tools and current-run state are enough. Add durable storage when work must survive a restart; add retrieval when the source collection outgrows a simple lookup; add a team when splitting responsibility helps the task. This keeps each new component tied to a concrete need.

1.3 Follow the Request End to End

The **agent loop** repeatedly uses an observation to choose the next step. A **pipeline** describes the data moving through stages; a **workflow** also describes branches, retries, coordination and stopping conditions.

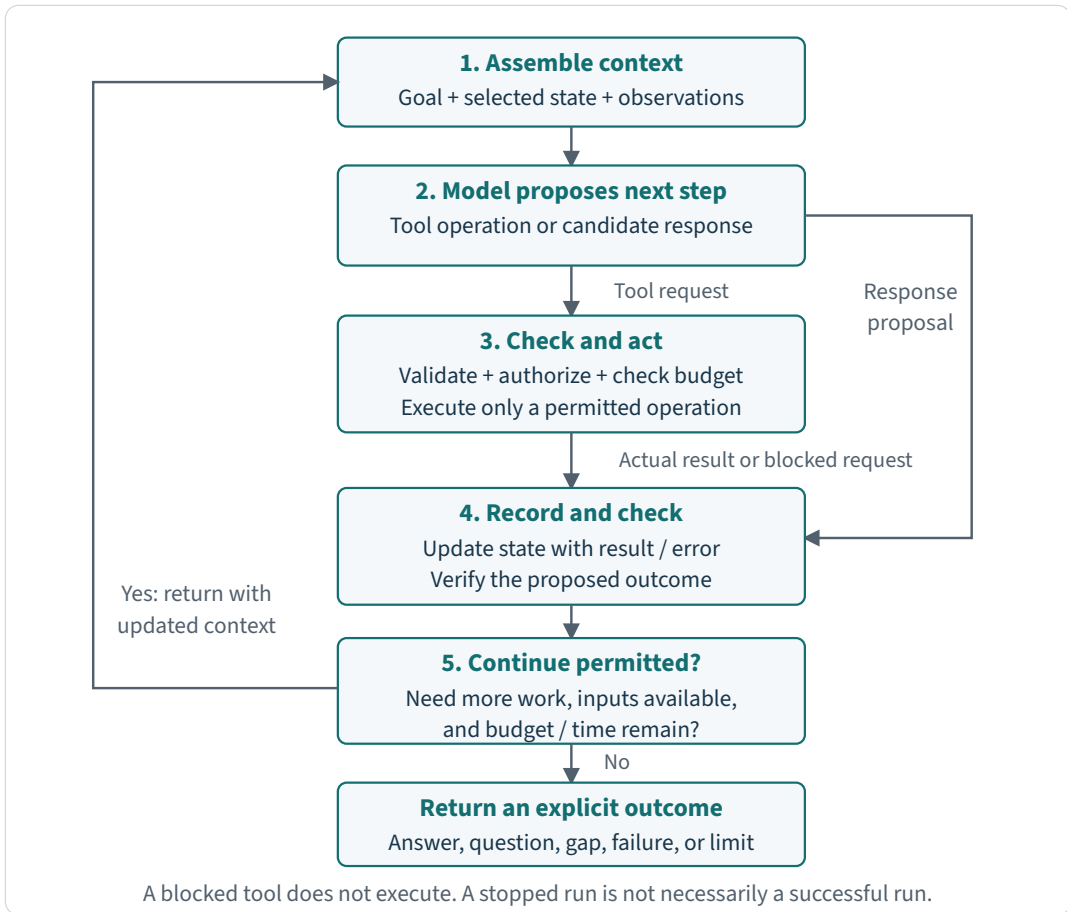


Figure 1.5: How the architecture runs over time. The runtime assembles context, the model proposes a tool operation or response, and actual results update state. The application checks outcomes and remaining limits before continuing. An answer, an evidence gap, and a budget stop are different outcomes.

1.3.1 Input, Decision, Action and Observation

First, the runtime combines the user's request with instructions and relevant state. The model then proposes a tool operation or a response. A tool request might look like this simplified JSON object:

```
{
  "tool": "read_source",
  "arguments": {"source_id": "travel-policy.txt"}
}
```

The runtime looks up `read_source`, checks `source_id` and executes the read. It returns the file contents—or the read error—to the model. OpenAI's function-calling guide specifies this request, execution and result exchange [3]; Chapter 4 implements it using native tool calls.

A **budget** limits work, such as the number of tool calls or elapsed time. The loop continues while another action is useful and permitted. It stops with an answer, a clarification request, missing information, an error or an exhausted limit. Recording the reason makes the run understandable when something goes wrong.

1.3.2 The Whole Application Workflow

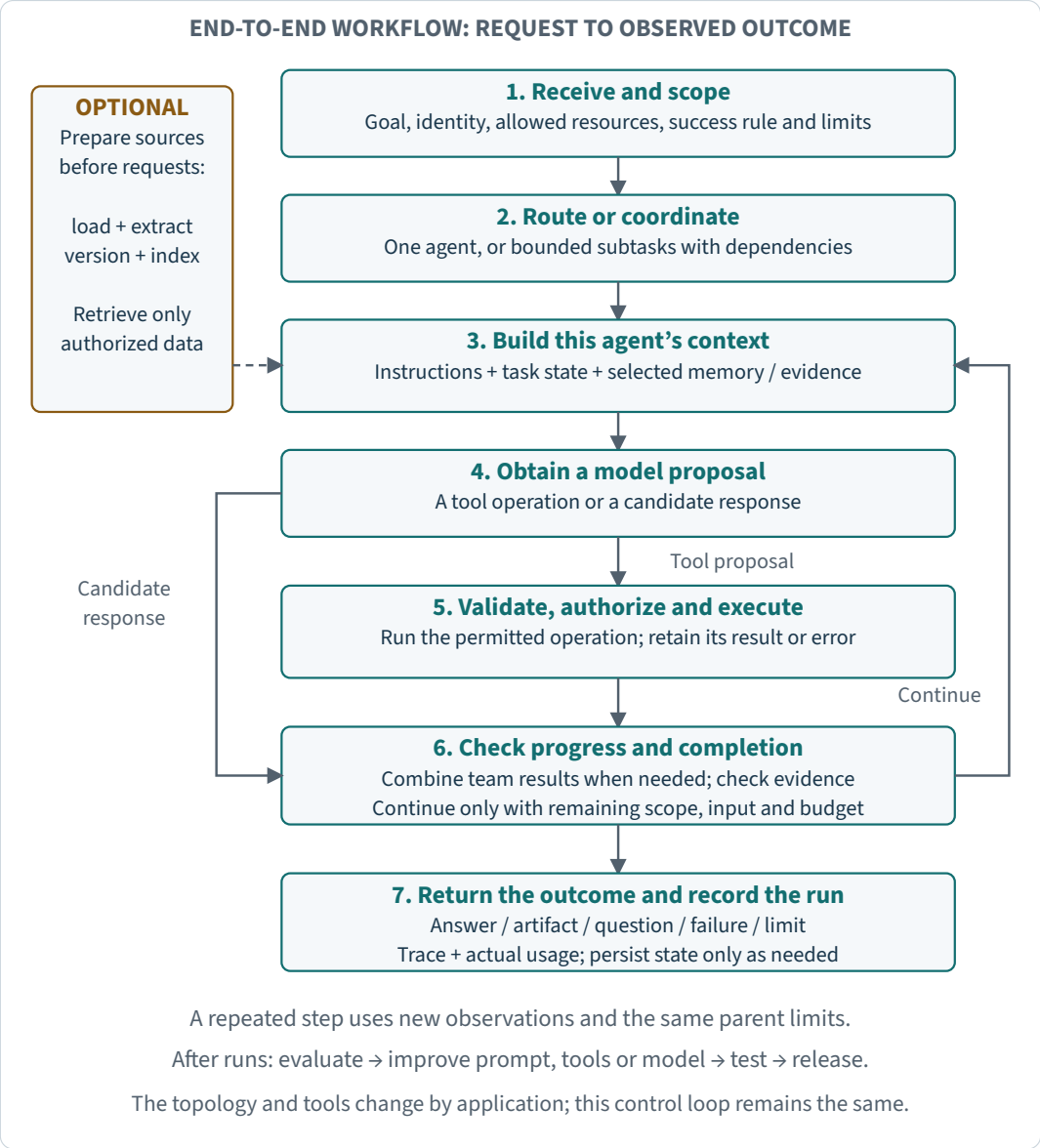


Figure 1.6: A request moves from task preparation through routing, context construction, a model proposal, permitted action and observed result. Candidate responses go directly to completion checks. Further work returns to context construction within the same task limits. Source indexing is an optional preparation path.

1. **Prepare the task.** Receive text, an event or media input; identify the goal, allowed resources and completion conditions.
2. **Route the work.** Start one agent or assign subtasks to several. Carry the parent task’s limits into delegated work.
3. **Build context and decide.** Select instructions, current observations and any retrieved material. Ask the model for the next action or response.
4. **Execute and observe.** Validate the request, check permission and run the tool. Add the returned result to the requesting agent’s state.
5. **Check and continue.** Complete the task, perform another step, request clarification or stop. A team may exchange results before this decision.
6. **Return and record.** Deliver the outcome and save the trace needed to diagnose the run. Evaluate representative runs before changing the application.

Documents can be indexed before requests arrive; saved memory can be retrieved during context construction. These are supporting paths, not mandatory stages of every agent. Training changes the model between evaluated releases and is covered separately in Chapter 18.

1.4 A Document Assistant in Action

Now apply the architecture to two fictional source documents:

Table 1.4: Sources available to the document assistant

Source ID	Contents
onboarding.txt	For travel expense limits, consult the current travel policy.
travel-policy.txt	Travel policy, effective 2026-01-01: the hotel limit is \$180 per night before tax. Exceptions require manager approval.

The user asks: **“What is the hotel limit, and are exceptions allowed?”** The answer needs the per-night amount, tax qualification, exception rule and source.

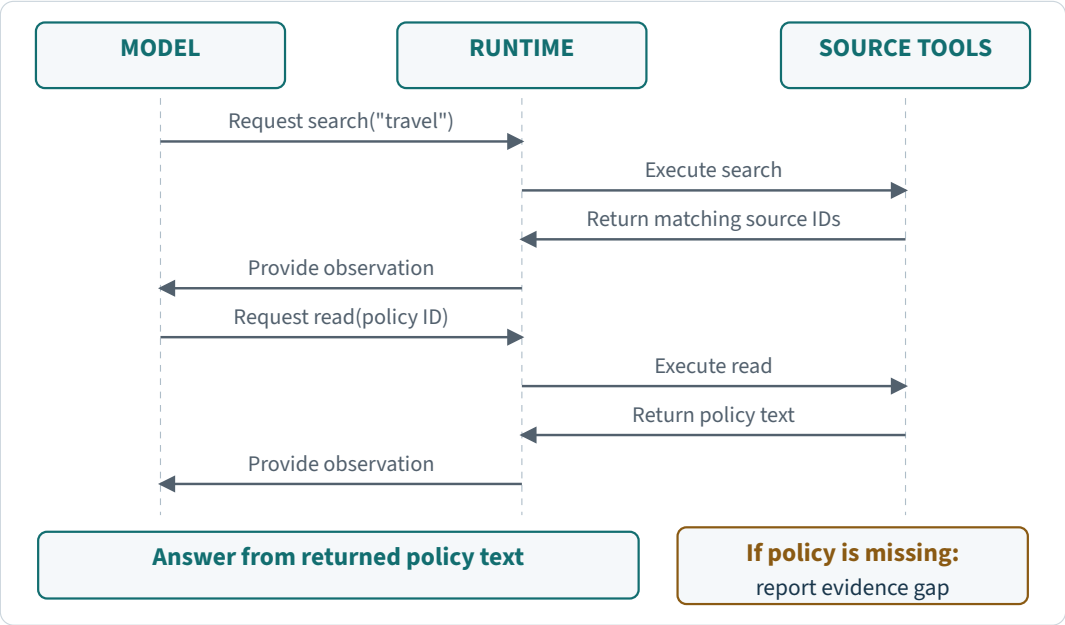


Figure 1.7: A possible research sequence. Search returns source IDs; read returns policy text; only then does the model answer from the observed text. A missing source leads to an explicit evidence gap.

The model first requests a search for travel information. Search returns source IDs. The model then requests `travel-policy.txt`, receives its text and can answer:

The policy allows \$180 per night before tax. Exceptions require manager approval. Source: `travel-policy.txt`.

If the policy is missing, the assistant should report that it could not find the rule. If the read fails, it should preserve the error so a retry or a useful failure message is possible. Search results identify what to read; the returned policy text supports the answer.

1.4.1 Where ReAct Fits

Worked reasoning examples improved results on selected arithmetic, commonsense and symbolic tasks in Wei and colleagues’ experiments [4]. Chapter 2 explains this prompting method; here we focus on how observations change the next action.

ReAct interleaves reasoning and actions: choose an operation, observe its result and use that observation in the next decision. Yao and colleagues reported benefits on the question-answering and interactive tasks they evaluated [5]. Our document loop illustrates the useful mechanism: reading a

source changes the information available for the next decision. Chapter 8 develops the implementation and comparison methods.

Table 1.5: Observable stages of a ReAct-style loop

Stage	Observable example
Decision	The question requires the contents of the supplied policy.
Action	<code>Requestread_source("travel-policy.txt").</code>
Observation	The tool returns the amount, qualification, date, and exception rule.
Next decision	Answer with those details and the source ID.

The table is an explanatory trace of the application, not a transcript of a model’s private reasoning. In code, inspect tool requests, returned data and task outcomes.

1.5 Decide Whether the Agent Is Worth Adding

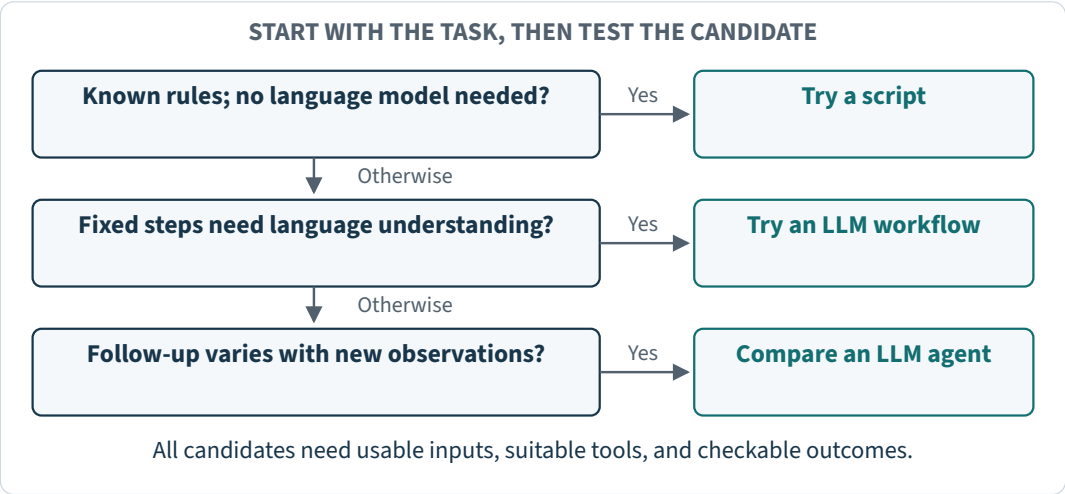


Figure 1.8: Choose the least complex candidate that meets the task requirements, then compare it on real cases. A model-directed loop is a candidate when follow-up steps vary and the task has usable tools and a way to check outcomes.

Compare candidates on the same questions, including missing or conflicting information. Use the following criteria to decide whether flexible action selection justifies its extra work:

Table 1.6: Criteria for comparing execution designs

Criterion	What it means	How to check it here
Known procedure	The useful steps can be written down in advance.	Try a fixed search/read/answer process on all test cases.

Criterion	What it means	How to check it here
Variable follow-up	Different observations require different next actions.	Include missing policies, references to other files, and conflicting rules.
Evidence access	The program can obtain the information needed for an answer.	Confirm that tools return actual IDs and source text.
Checkable outcome	Someone or something can distinguish a supported result from an unsupported one.	Compare each answer with the supplied documents and task requirements.
Latency	The elapsed time from request to usable result.	Measure total duration, in seconds, over repeated runs.
Cost	The resources charged or consumed by a run.	Record model/tool charges where available; Chapter 2 explains the accounting.
Failure impact	The consequence of a wrong answer or operation.	Begin with a read-only assistant and inspect unsupported claims.

Measure completed tasks, failures, latency and cost. Repeat model-based runs because outputs can vary. The smallest implementation that meets the requirements is a useful starting point.

1.6 Run a Small Version of the Loop

This Python demonstration uses written decision rules so the execution sequence is easy to inspect. It makes no model calls. Chapter 4 replaces the rule-based selector with an LLM while retaining the runtime’s responsibility for tools, state and stopping.

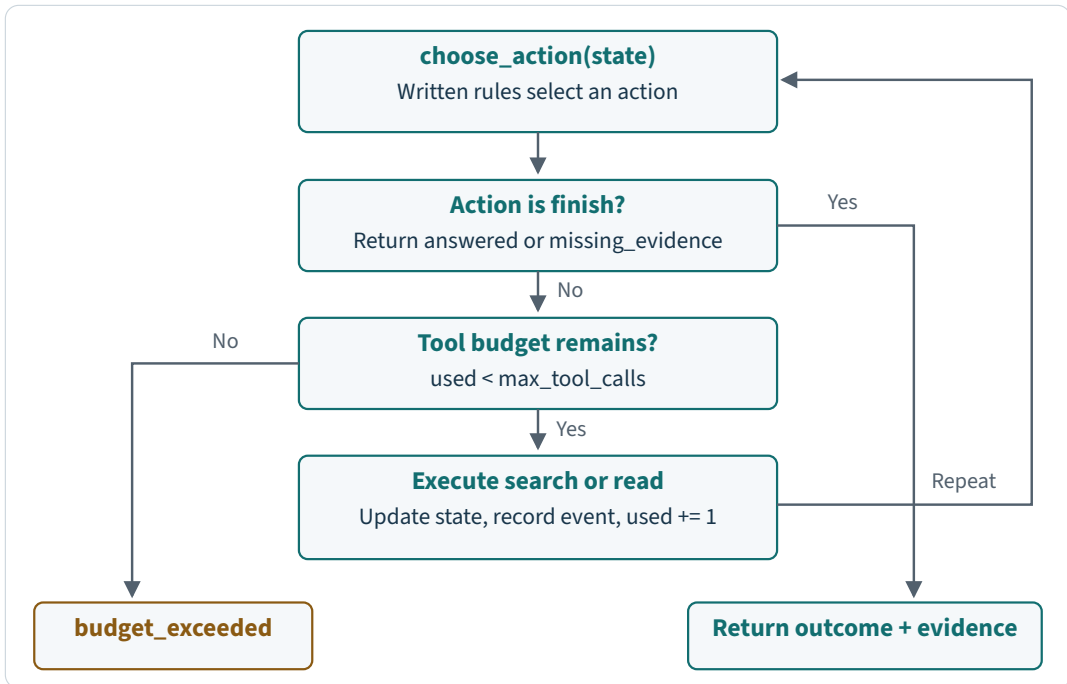


Figure 1.9: The rule-based demonstration's control flow. `choose_action` inspects state; `run_demo` executes a search or read, records the result, and loops. The terminal outcomes are answered, missing_evidence, and budget_exceeded.

`choose_action` proposes a step; `run_demo` executes it and updates state. The comments explain the budget and stopping decisions. Save the complete program as `code-examples/chapter-01/loop_demo.py`, or use the supplied file.

```
# Runnable: code-examples/chapter-01/loop_demo.py
"""Offline loop demo: written rules; no model calls."""

SOURCES = {
    "onboarding.txt": (
        "For travel expense limits, consult "
        "the current travel policy."
    ),
    "travel-policy.txt": (
        "Travel policy, effective 2026-01-01: "
        "the hotel limit is "
        "$180 per night "
        "before tax. Exceptions require manager approval."
    ),
}

def choose_action(state):
    if "matches" not in state:
        return "search", "travel"
```

```

    if "travel-policy.txt" not in state["matches"]:
        return "finish", "missing_evidence"
    if "policy" not in state:
        return "read", "travel-policy.txt"
    return "finish", "answered"

def run_demo(sources, max_tool_calls=2):
    if type(max_tool_calls) is not int or max_tool_calls < 0:
        raise ValueError(
            "max_tool_calls must be a nonnegative integer"
        )
    state = {}
    events = []
    used = 0
    while True:
        action, argument = choose_action(state)
        # Finishing consumes no tool call; check it before the
budget.
        if action == "finish":
            return argument, state.get("policy"), events
        if used >= max_tool_calls:
            return "budget_exceeded", None, events
        if action == "search":
            matches = []
            for source_id, text in sources.items():
                if argument.lower() in text.lower():
                    matches.append(source_id)
            state["matches"] = matches
            events.append(f"SEARCH: {len(matches)} source(s)")
        elif action == "read":
            state["policy"] = sources[argument]
            events.append(f"READ: {argument}")
            # The next decision reads the observation just saved in
state.
            used += 1

if __name__ == "__main__":
    missing = {"onboarding.txt": SOURCES["onboarding.txt"]}
    cases = [
        ("Available policy", SOURCES, 2),
        ("Missing policy", missing, 2),
        ("One-call budget", SOURCES, 1),
    ]
    for label, sources, budget in cases:
        status, text, events = run_demo(sources, budget)
        print(f"\n{label}: {status}")
        for event in events:
            print(f" {event}")
        if text is not None:
            print(f" EVIDENCE: {text}")

```

Run it from the companion-package root:

```
python code-examples/chapter-01/loop_demo.py
```

OFFLINE DEMONSTRATION · RECORDED OUTPUT

Three actual local runs; no model or API calls.

```
Available policy: answered
  SEARCH: 2 source(s)
  READ: travel-policy.txt
  EVIDENCE: Travel policy, effective 2026-01-01: the hotel limit is
$180 per night before tax. Exceptions require manager approval.

Missing policy: missing_evidence
  SEARCH: 1 source(s)

One-call budget: budget_exceeded
  SEARCH: 2 source(s)
```

Read the status first, then the trace. A search result is not a completed read.

Figure 1.10: Recorded output from the offline demonstration. The available-policy run reads the source; the missing-policy run reports missing evidence; the one-call budget ends after search, before reading.

The first run searches and reads, then returns `answered` with the policy text. The second returns `missing_evidence` because the policy is absent. The third returns `budget_exceeded` after its one permitted search. These three outputs make the relationship between available data, permitted work and outcome visible.

Table 1.7: Run outcomes and what to communicate

Outcome	What the system knows	What it should communicate
Supported answer	Relevant source text was obtained and the answer matches the contract.	The answer, scope, and supporting source.
Missing evidence	Required information was not found in the available material.	The specific gap and what would be needed.
Conflicting evidence	Retrieved sources do not establish one applicable answer.	The disagreement and unresolved scope.
Tool failure	An operation did not produce the required result.	What failed and what remains unknown.
Budget exhausted	A configured resource limit prevents more work.	Progress made and the unfinished task.

References

- [1] Russell S, Norvig P. Intelligent agents: chapter preview for Artificial Intelligence: A Modern Approach. <https://aima.cs.berkeley.edu/4th-ed/pdfs/newchap02.pdf> (accessed September 15, 2026).
- [2] Anthropic. Building effective agents 2024. <https://www.anthropic.com/engineering/building-effective-agents> (accessed September 15, 2026).
- [3] OpenAI. Function calling. <https://developers.openai.com/api/docs/guides/function-calling> (accessed September 15, 2026).
- [4] Wei J, Wang X, Schuurmans D, Bosma M, Ichter B, Xia F, et al. Chain-of-thought prompting elicits reasoning in large language models. arXiv 2022. <https://doi.org/10.48550/arXiv.2201.11903>.
- [5] Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, et al. ReAct: Synergizing reasoning and acting in language models. arXiv 2023. <https://doi.org/10.48550/arXiv.2210.03629>.